

Self-Reflective Report

BIT1123 Object Oriented Programming — Assignment 1

Student Name: Alfatih Mohammed Salih Ali

Student ID: 202504010010

Course: BIT1123 Object Oriented Programming

Institution: City University Malaysia

1. Introduction

This report reflects on the weekly lab tutorials completed for BIT1123 Object Oriented Programming, from the first week of setting up a GitHub repository through to the Week 10 graphical quiz application. The purpose of the assignment is not only to collect the code in one place, but to look back at what each tutorial was actually teaching and to be honest about which parts were understood immediately and which parts only made sense later.

Looking at the tutorials as a sequence rather than as ten separate lab sessions makes the design of the course visible. Week 1 was about tools. Week 2 introduced the class. Weeks 3 to 7 added one object oriented concept at a time — inheritance, encapsulation, protected inheritance, abstraction. Weeks 8 and 9 moved to collections and files, and Week 10 combined everything into a program with a user interface. Each week assumed the previous one had been understood, which meant that a gap in one week showed up as confusion two weeks later.

This report covers what was learned in each tutorial, the specific problems that came up in the code, how they were fixed, and what still needs work.

2. Knowledge Gained from Each Tutorial

Week 1 — Repository setup and first Java programs

The first tutorial was more about the development environment than about Java. It covered creating a GitHub account and repository, adding a `.gitignore` for Java and an MIT licence, opening the project in GitHub Codespaces, and using the basic git commands: `git init`, `git status`, `git add`, `git commit -m` and `git push`.

The Java side was still procedural. `HelloWorld.java` was a single class with a single main method. `StudentGrade.java` went further: it used `Scanner` to read a student's name, ID, course and three subject marks, calculated the total and average, and used a chain of `if/else if` statements to convert the average into a letter grade. `String.format("%.2f", average)` was used to print the average to two decimal places.

What this week established was the habit of committing work rather than leaving it on one machine. The value of that was not obvious at the time, and the consequences of not doing it properly appear later in this report.

Week 2 — Classes, constructors and objects

This was the week the course actually changed direction. Instead of writing everything inside `main`, the work was split between two files. `Student.java` declared the attributes `name`, `age` and `gpa`, a constructor that assigned them, and methods `displayInfo()` and `study()`. `Main.java` only created the object with `new Student(...)` and called its methods.

The idea that took a moment to settle was that the constructor parameters (`studentName`, `studentAge`, `studentGpa`) are different names from the fields they assign to. Once that was clear, the `study()` method made the point of the whole exercise: calling `s1.study()` changed the object's internal `gpa`, and calling `s1.displayInfo()` afterwards showed a different value. The object was holding state between method calls, which a loose collection of variables in `main` does not do in the same way.

Weeks 3–4 — Inheritance, overriding and polymorphism

The scenario was a university with different kinds of people. `Person` held `name` and `id` as private fields with `getName()` and `getId()` to read them, and an `introduce()` method printing "I am a person." `Student` and `Lecturer` both used `extends Person`, called `super(name, id)` in their constructors, and used `@Override` to replace `introduce()` with their own version.

The part worth reflecting on is the `Main` class. All three objects are declared as `Person`:

```
Person p1 = new Person(...);
Person p2 = new Student(...);
Person p3 = new Lecturer(...);
```

and yet `p2.introduce()` prints "I am a student." The reference type is `Person`, but the method that runs is chosen by the object that was actually created. This is runtime polymorphism, and it is much easier to see in this three-line example than to understand from a definition.

`super(name, id)` also needed thinking about. Because `name` and `id` are private in `Person`, the subclass cannot assign them directly; it has to hand the values to the parent constructor and let `Person` store them.

Week 5 — Encapsulation

This tutorial made the access modifiers the actual subject rather than a detail. The scenario was a student information system where students must not be able to change their own records. `Student` declared `studentID`, `name`, `cgpa` and `programme` as private, with a `setX()` and `getX()` for each, and `programme` was added as the extra variable required by the activity.

The written questions in `documentation.txt` were the useful part. Explaining **why** the fields are private forces a clearer answer than just typing the keyword. The key realisation was that a setter is not a pointless wrapper around an assignment: it is the only place where a rule can be enforced. `setCGPA()` is where a check that the value falls between 0.00 and 4.00 would go. If the field were public, there would be nowhere to put that check, because nothing would have to go through the class at all.

Week 6 — Protected inheritance and constructor chaining

Week 6 revisited inheritance with a different access modifier. `Employee` declared `id` and `name` as protected rather than private, and `Lecturer` extends `Employee` added `subject` and, as required by the activity, `department`.

Comparing this week directly against Weeks 3–4 was what made the modifiers make sense. In Weeks 3–4 the base class fields were private, so the subclass could only reach them through getters. Here they are protected, so `Lecturer` could use `id` and `name` directly if it needed to. That is more convenient, but it also means the base class no longer fully controls its own data — every subclass, including ones written later, can change those fields without going through `Employee`.

`super(id, name)` also appeared again, and this time it was clear why it must be the first statement in the constructor: the parent part of the object has to exist before the child part can be added to it.

Week 7 — Abstraction

The smart home scenario asked for a system where every appliance can turn on, turn off and operate, but where each appliance operates differently. `Appliance` was written as an abstract class with concrete `turnOn()`, `turnOff()` and `displayBrand()` methods shared by everything, and `public abstract void operate();` with no body at all.

The declaration that a method has no body was the new idea. `abstract` is a promise enforced by the compiler: any class extending `Appliance` must supply `operate()` or it will not compile. The two appliances added were `WashingMachine`, printing "Washing clothes...", and `Television`, printing "Displaying TV channels...".

This also completed the picture from Weeks 3–4. Abstraction is what makes polymorphism worth using: because every appliance is guaranteed to have `operate()`, code can call it on any appliance without knowing or caring which one it has.

Weeks 8–9 — Collections, file input and output

These weeks moved away from designing classes and towards handling data. The task manager program used `ArrayList<String>` instead of a fixed-size array, which meant `tasks.add(task)` could be called without deciding the size in advance, and `tasks.size()` and `tasks.get(i)` were used to read it back.

The second half was persistence. `BufferedWriter` wrapped around a `FileWriter` wrote each task to `task.txt`, and `BufferedReader` wrapped around a `FileReader` read the file back line by line until `readLine()` returned `null`. Both were written using `try-with-resources`:

```
try (java.io.BufferedWriter writer = new java.io.BufferedWriter(new java.io.FileWriter("task.txt"))) {
```

so the file is closed automatically even if an error occurs, with a `catch` block for `IOException`. Understanding that file operations are **checked** exceptions — that the compiler refuses to build the program unless the failure is handled — was new. Up to this point, code that compiled was code that ran.

Week 10 — Swing GUI and event-driven programming

The final tutorial built the Programming Quiz Battle, a windowed application. `Questions` was a plain data class holding the question text, two options and the correct answer, with getters and an `isCorrect(String choice)` method. `QuizBattleGUI` extended `JFrame`, implemented `ActionListener`, and built the interface from a `JLabel` for the question, two `JButton` objects and a `JLabel` for the result, each positioned with `setBounds()` after `setLayout(null)`.

The genuinely new concept was event-driven execution. Every program before this ran from the first line of `main` to the last and then stopped. Here, the constructor builds the window and then `main` finishes, but the program keeps running and waits. `actionPerformed(ActionEvent e)` is not called by any line of code that was written — it is called by Java when the user clicks. Working out that `e.getSource()` identifies which button was pressed, and that it has to be cast back to `JButton` to read its text, took the longest part of the tutorial.

The design point is also worth noting: `Questions` knows nothing about windows, and `QuizBattleGUI` does not decide what the right answer is — it asks `question.isCorrect(...)`. That separation between data and interface is the same idea introduced in Week 2, just applied to a bigger program.

3. Challenges Encountered

The difficulties in this subject fell into three groups, and only one of them was about Java syntax.

Broken code that the compiler caught. The clearest example is in the Week 2 folder. It ended up containing two files, `Main.java` and `MIN.JAVA`, both declaring `public class Main`. This produces two separate problems. First, `javac` does not accept a file with a `.JAVA` extension as a source file at all and rejects it with `invalid flag`. Second, even after renaming it to a valid `.java` name, the compiler reports `class Main is public, should be declared in a file named Main.java`. The folder could not be compiled as a whole in the state it was committed in. The rule that a public class name must exactly match its filename had been treated as a formality rather than something the compiler enforces.

Design decisions that the compiler does not catch. These were harder, because the code ran and still was not right. Choosing between `private` and `protected` in Weeks 5 and 6 was the main one: both compile, both work, and the difference only matters for whether the base class keeps control of its own data. The Week 7 `Main` is another example — it declares the appliances as `WashingMachine wm` and `Television tv` rather than as `Appliance`, which works perfectly but does not use the polymorphism the abstract class was designed to enable. Understanding why one version is better than the other required going back to Weeks 3–4 rather than looking at any error message.

Organisation and version control. This was the largest problem and it had nothing to do with Java. The tutorial work ended up scattered across four separate GitHub repositories instead of the single repository the subject required: the Week 7 appliance work sat in one repository, the Week 1 grade calculator in another, and Weeks 2 and 8–9 in a third, with a fourth left empty. Alongside that, the commit messages were `Initial commit`, `branch 2` and `tasks1`, none of which say what changed. A generated output file, `task.txt`, was committed to the repository root rather than beside the code that produced it, so the root of the project contained a list of hobbies with no visible connection to anything.

Assembling this assignment meant first working out what existed and where, which would not have been necessary if the work had been committed properly the first time.

4. How the Challenges Were Overcome

The duplicate-class problem was fixed by keeping one entry point. `MIN.JAVA` and `Main.java` differed only in the data passed to the constructor, so the version carrying the correct student details was kept, renamed properly to `Main.java`, and the other deleted. The folder now compiles. The general lesson was applied everywhere else: every public class in this repository is in a file with exactly its own name, checked by compiling each folder rather than assuming.

The design questions were resolved by comparing weeks rather than reading definitions again. Placing the Week 5 `Student` class next to the Week 6 `Employee` class makes the effect of `private` versus `protected` concrete, because the same relationship is written both ways. The same approach worked for polymorphism: writing the Weeks 3–4 `Main` with all three objects held in `Person` references, and seeing three different lines of output from three identical method calls, settled the concept better than any explanation.

The organisational problem was fixed by consolidating everything. The Week 7 files and the Week 1 grade calculator were brought out of their separate repositories into this one, so the whole subject now lives in a single place with the folder structure the assignment specifies. A branch structure was adopted — `main` for the finished submission, `develop` for integration, and a short-lived `tutorial/weekN` branch for each week — and every commit message now states which week it belongs to and what changed. `task.txt` was moved into `week8-9/` next to the program that writes it.

The most reliable fix, though, was simply running the code. Every folder in this repository was compiled with `javac *.java` and executed, and the output compared against the expected output printed in the tutorial handout. Weeks 3–4, 5 and 6 match their handouts exactly. That check would have caught the Week 2 problem the day it was committed.

5. Improvements in Java Programming Skills

The most measurable improvement is in how a program gets structured. The Week 1 grade calculator is a single class with everything inside `main`: input, calculation, decision and output in one block. By Week 7 the same amount of logic is spread across an abstract class and two subclasses, with `main` doing almost nothing except creating objects and calling methods on them. The instinct changed from "add more lines" to "which class should be responsible for this".

Specific skills gained along the way:

- bullet **Access modifiers used deliberately.** `private`, `protected` and `public` are now chosen based on who should be allowed to reach a field, not copied from an example.
- bullet **Constructors and `super()`.** Passing values up a class hierarchy, and understanding why `super()` has to come first.

- bullet **@Override used properly.** It is not decoration — it makes the compiler verify that a method really is overriding something, which catches a misspelled method name that would otherwise silently do nothing.
- bullet **Exception handling.** Checked exceptions and try-with-resources from Weeks 8–9 were the first time the compiler demanded that failure be planned for.
- bullet **Collections.** `ArrayList` and its methods, and why it is preferable to a fixed-size array when the number of items is not known.
- bullet **Reading compiler errors.** `class Main is public, should be declared in a file named Main.java` is a precise instruction, not noise. Learning to read the message rather than guess at the cause saved a lot of time.
- bullet **Git as a working tool.** Branching, meaningful commit messages, and keeping generated files out of source folders.

Areas that are still weak: input validation is absent from every program written so far — `StudentGrade` will crash on non-numeric input because there is no check around `nextDouble()`. Nothing written uses interfaces, `ArrayList` of custom objects, or more than one question in the Week 10 quiz.

6. Understanding of Object-Oriented Programming

The four pillars are each demonstrated by a specific tutorial in this repository, which makes them easier to describe concretely than in the abstract.

Encapsulation (Week 5). An object's data is private and reachable only through methods the class provides. The point is control: the class decides what may be read, what may be changed, and under what conditions. Without it there is no place to enforce a rule, because nothing has to pass through the class.

Inheritance (Weeks 3–4, 6). A specialised class is built on a general one so shared code is written once. `Student` and `Lecturer` do not repeat the storage of `name` and `id`; they inherit it and add only what makes them different. The two weeks show it from both sides — `private` fields with getters keeps the parent in control, protected fields give subclasses direct access and give up some of that control.

Polymorphism (Weeks 3–4, 7). One reference type behaves differently depending on the object it actually holds. Three `Person` references produce three different introductions, and the calling code does not need to know which is which. This is the pillar that changes how code is written, because it means a program can be extended by adding a new subclass rather than by adding another branch to an `if` statement.

Abstraction (Week 7). A base class defines what its subclasses must be able to do without saying how. `Appliance` guarantees that everything extending it has `operate()`, which is exactly the guarantee that makes polymorphism safe to rely on.

The broader understanding is that these four are not four separate techniques. Encapsulation makes an object responsible for its own data, inheritance lets that responsibility be shared, abstraction states what a group of classes must promise, and polymorphism is what that promise is for. The Week 10 program

is the small proof of this: `Questions` owns the answer and decides whether a choice is correct, and `QuizBattleGUI` only displays things and passes clicks along. Neither class does the other's job.

7. Future Learning Plans

The immediate gap is that nothing written so far handles bad input. The first goal is to go back through these programs and add validation — wrapping `Scanner` input so that a non-numeric mark is rejected and re-prompted rather than crashing, and adding range checks inside the Week 5 setters so that `setCGPA()` actually enforces the 0.00 to 4.00 rule it was designed to allow.

Beyond that, the plan is:

- bullet **Interfaces.** Abstract classes were covered in Week 7, but interfaces and the difference between the two were not. Understanding when a class should implement a contract rather than extend a parent is the obvious next step.
 - bullet **Collections of objects.** Every `ArrayList` used so far has held `String`. Storing `ArrayList<Student>` and sorting or searching it is a small change with a large increase in what a program can do.
 - bullet **Extending the Week 10 application.** The quiz currently has one hard-coded question. Loading a list of `Questions` objects, moving through them, and keeping score would combine the GUI work with collections and file handling from Weeks 8–9 into one project.
 - bullet **Exception handling in more depth.** Custom exception classes, and the difference between checked and unchecked exceptions.
 - bullet **Better development practice.** Learning to write simple tests rather than checking output by eye, and continuing to use branches and clear commit messages, since the organisational problems in this subject cost more time than any syntax error did.
 - bullet **Reading the official documentation.** Getting comfortable with the Java API documentation rather than relying on examples, so that unfamiliar classes can be worked out independently.
-

8. Conclusion

The ten tutorials moved from a single procedural class to a small event-driven application built from cooperating objects, and the progression was deliberate rather than accidental. Each week added one idea and relied on the previous one being understood.

The most useful thing learned was not any single keyword. It was that in object oriented programming, the important decisions are the ones the compiler cannot check. `private` and `protected` both compile. Declaring a variable as `WashingMachine` or as `Appliance` both compile. Putting all the logic in `main` or splitting it across classes both compile. The difference only shows up later, when the program has to be extended or corrected, and by then the decision has already been made.

The other lesson came from assembling this repository rather than from any tutorial. Work spread across four repositories, commit messages that say nothing, a file that will not compile because of its

own name, and a generated file committed to the wrong folder — none of these were difficult problems, but together they took longer to sort out than any of the programming did. Version control is only useful if it is used carefully, and that is a habit rather than a skill.

The repository submitted with this report now holds every tutorial in one place, in the required structure, with each folder compiling and running correctly.

9. GitHub Repository URL

Repository:

https://github.com/qwasdfkl80-ship-it/ALFATIH-MOHAMMED-SALIH-ALI_BIT1123-OOP-Assignment-1

The complete submission is on the `main` branch.